Assignment 2 – Natural Language Processing

SIMATS School of Engineering (Approved by AICTE | IET-UK Accreditation)

Course Code: DSA03 **Course Name:** Natural Language Processing

Branch: CSE **Year:** 2026

Student Name:V.Poojya lakshmi

Register No.: 192324273

Questions covered: Q1 – Add-1 (Laplace) smoothing of bigram counts (10M, CO2, BLT4); Q2 – Bag generation using an N-gram model (10M, CO2, BLT4); Q3 – Penn Treebank POS tagging of Damon Runyon sentences (10M, CO2, BLT4)

Question 1 – Add-1 Smoothing of the Berkeley Restaurant Project Bigrams

Question. Figure 4.1 shows the bigram counts from a piece of a bigram grammar from the Berkeley Restaurant Project. Note that the majority of the values are zero. Perform the add-1 smoothed counts for the bigram probability values.

N-gram models and the zero-count problem

A language model assigns a probability to a sequence of words. By the chain rule, P(w1 w2 ... wn) = P(w1) · P(w2 | w1) · P(w3 | w1 w2) ... P(wn | w1 ... wn-1). Because long histories are almost never seen in a corpus, an N-gram model uses the Markov assumption: the probability of a word depends only on the previous N-1 words. For a bigram (N = 2) model:

P(wn | wn-1) ≈ C(wn-1 wn) / C(wn-1)

Here C(wn-1 wn) is the number of times the bigram occurs in the training corpus and C(wn-1) is the number of times the first word occurs. This is the **Maximum Likelihood Estimate (MLE)**, because it gives the highest probability to the training data.

The weakness of MLE is **sparsity**. A corpus is finite, so many perfectly grammatical bigrams never occur in it. MLE gives each of them probability exactly zero. Since the probability of a sentence is a product of bigram probabilities, a single unseen bigram makes the whole sentence probability zero. It also makes perplexity undefined (division by zero) when the model is evaluated on test data. The standard remedy is **smoothing** (also called discounting): take a little probability mass from the seen events and give it to the unseen ones.

1 (Laplace) smoothing

Laplace smoothing is the simplest method. Before normalising, we pretend that every possible bigram occurred one more time than it actually did, so no count is ever zero.

Smoothed probability:

P*Laplace(wn | wn-1) = (C(wn-1 wn) + 1) / (C(wn-1) + V)

The numerator gets +1. The denominator gets +V because there are V possible next words and each of them received an extra count of 1, so the total for the history word grows by V. In the Berkeley Restaurant Project corpus (9332 sentences) the vocabulary size is **V = 1446** word types.

It is also useful to see what add-1 does to the counts themselves. The **reconstructed (adjusted) count** is the count that, divided by the original C(wn-1), would give the smoothed probability:

$c^*(w_{n-1} w_n) = (C(w_{n-1} w_n) + 1) \cdot C(w_{n-1}) / (C(w_{n-1}) + V)$

The **discount ratio** $d_c = c^* / c$ tells us what fraction of the original count survives.

Data given

Figure 4.1 – Bigram counts for eight words (out of V = 1446) in the Berkeley Restaurant Project corpus of 9332 sentences. Rows are the previous word, columns are the next word.

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Out of the 64 cells in this table, 31 are zero, so almost half of the cells are zero even for these very common words. Across the whole vocabulary (1446 × 1446 possible bigrams) the proportion of zeros is far higher.

To normalise we also need the **unigram counts** of the same corpus:

Word	i	want	to	eat	chinese	food	lunch	spend
C(w)	2533	927	2417	746	158	1093	341	278
C(w) + V	3979	2373	3863	2192	1604	2539	1787	1724

The second row, C(w) + V, is the new denominator for every cell in that word's row after add-1 smoothing (for example 2533 + 1446 = 3979 for the row of 'i').

Step 1 – Add-1 smoothed counts

We add 1 to every cell of Figure 4.1. Every zero becomes 1 and every other count increases by 1.

Figure 6.1 – Add-1 smoothed bigram counts, C + 1

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Step 2 – Normalise to get the add-1 bigram probabilities

Each cell is divided by the row's new total, C(wn-1) + V. The row denominators are 3979, 2373, 3863, 2192, 1604, 2539, 1787 and 1724 respectively.

Worked examples

- P(want | i) = (827 + 1) / (2533 + 1446) = 828 / 3979 = **0.20809**
- P(to | i) = (0 + 1) / (2533 + 1446) = 1 / 3979 = **0.00025** (was 0)
- P(to | want) = (608 + 1) / (927 + 1446) = 609 / 2373 = **0.25664**
- P(eat | to) = (686 + 1) / (2417 + 1446) = 687 / 3863 = **0.17784**
- P(spend | to) = (211 + 1) / 3863 = 212 / 3863 = **0.05488**
- P(lunch | eat) = (42 + 1) / (746 + 1446) = 43 / 2192 = **0.01962**
- P(food | chinese) = (82 + 1) / (158 + 1446) = 83 / 1604 = **0.05175**
- P(i | food) = (15 + 1) / (1093 + 1446) = 16 / 2539 = **0.00630**
- P(food | lunch) = (1 + 1) / (341 + 1446) = 2 / 1787 = **0.00112**
- P(chinese | spend) = (0 + 1) / (278 + 1446) = 1 / 1724 = **0.00058** (was 0)

*Figure 6.2 – Add-1 smoothed bigram probabilities, P(column | row)**

	i	want	to	eat	chinese	food	lunch	spend
i	0.00151	0.20809	0.00025	0.00251	0.00025	0.00025	0.00025	0.00075
want	0.00126	0.00042	0.25664	0.00084	0.00295	0.00295	0.00253	0.00084
to	0.00078	0.00026	0.00129	0.17784	0.00078	0.00026	0.00181	0.05488
eat	0.00046	0.00046	0.00137	0.00046	0.00776	0.00137	0.01962	0.00046
chinese	0.00125	0.00062	0.00062	0.00062	0.00062	0.05175	0.00125	0.00062
food	0.00630	0.00039	0.00630	0.00039	0.00079	0.00197	0.00039	0.00039

	i	want	to	eat	chinese	food	lunch	spend
lunch	0.00168	0.00056	0.00056	0.00056	0.00056	0.00112	0.00056	0.00056
spend	0.00116	0.00058	0.00116	0.00058	0.00058	0.00058	0.00058	0.00058

For comparison, here are the **unsmoothed (MLE) probabilities** computed from Figure 4.1 as C(wn-1 wn) / C(wn-1):

Figure 6.3 – Unsmoothed bigram probabilities

	i	want	to	eat	chinese	food	lunch	spend
i	0.0020	0.3265	0.0000	0.0036	0.0000	0.0000	0.0000	0.0008
want	0.0022	0.0000	0.6559	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.0008	0.0000	0.0017	0.2838	0.0008	0.0000	0.0025	0.0873
eat	0.0000	0.0000	0.0027	0.0000	0.0214	0.0027	0.0563	0.0000
chinese	0.0063	0.0000	0.0000	0.0000	0.0000	0.5190	0.0063	0.0000
food	0.0137	0.0000	0.0137	0.0000	0.0009	0.0037	0.0000	0.0000
lunch	0.0059	0.0000	0.0000	0.0000	0.0000	0.0029	0.0000	0.0000
spend	0.0036	0.0000	0.0036	0.0000	0.0000	0.0000	0.0000	0.0000

1.6 Step 3 – Reconstructed counts c*

To see how much the smoothing has changed the original data, we convert the smoothed probabilities back into counts using c* = (C + 1) · C(wn-1) / (C(wn-1) + V).

Worked examples

- c*(i want) = (827 + 1) × 2533 / 3979 = 828 × 0.63662 = **527.1**
- c*(want to) = (608 + 1) × 927 / 2373 = 609 × 0.39064 = **237.9**
- c*(to eat) = (686 + 1) × 2417 / 3863 = 687 × 0.62568 = **429.8**
- c*(chinese food) = (82 + 1) × 158 / 1604 = 83 × 0.09850 = **8.2**
- c*(i to) = (0 + 1) × 2533 / 3979 = **0.64** (was 0)

Figure 6.4 – Add-1 reconstructed counts c*

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527.1	0.6	6.4	0.6	0.6	0.6	1.9
want	1.2	0.4	237.9	0.8	2.7	2.7	2.3	0.8
to	1.9	0.6	3.1	429.8	1.9	0.6	4.4	132.6
eat	0.3	0.3	1.0	0.3	5.8	1.0	14.6	0.3

	i	want	to	eat	chinese	food	lunch	spend
chinese	0.2	0.1	0.1	0.1	0.1	8.2	0.2	0.1
food	6.9	0.4	6.9	0.4	0.9	2.2	0.4	0.4
lunch	0.6	0.2	0.2	0.2	0.2	0.4	0.2	0.2
spend	0.3	0.2	0.3	0.2	0.2	0.2	0.2	0.2

1.7 Discount analysis

The discount ratio d_c = c* / c shows how much of each observed count is lost to the unseen bigrams.

Bigram	Original c	Smoothed c*	d_c = c*/c	MLE probability	Add-1 probability
i want	827	527.1	0.64	0.3265	0.20809
want to	608	237.9	0.39	0.6559	0.25664
to eat	686	429.8	0.63	0.2838	0.17784
to spend	211	132.6	0.63	0.0873	0.05488
eat lunch	42	14.6	0.35	0.0563	0.01962
chinese food	82	8.2	0.10	0.5190	0.05175
food to	15	6.9	0.46	0.0137	0.00630
i to	0	0.64	–	0.0000	0.00025

1.8 Effect on a sentence probability

Consider the fragment 'i want to eat chinese food'. Using the bigrams i→want, want→to, to→eat, eat→chinese and chinese→food:

- **Unsmoothed:** 0.3265 × 0.6559 × 0.2838 × 0.0214 × 0.5190 = **6.77 × 10^-4**
- **Add-1 smoothed:** 0.20809 × 0.25664 × 0.17784 × 0.00776 × 0.05175 = **3.81 × 10^-6**

Now consider 'i want to eat lunch chinese', which contains the bigram lunch→chinese. That bigram has count 0 in Figure 4.1, so the unsmoothed probability of the fragment is **exactly 0**. After smoothing it becomes about **1.04 × 10^-7**: small, as it should be for an unlikely word order, but no longer impossible. This is the practical purpose of smoothing.

Observations

1. **No zero remains.** Every cell now has a non-zero probability, so a sentence containing a previously unseen bigram no longer gets probability 0 and perplexity can be computed.

2. **Probability mass was moved from seen to unseen bigrams.** Each row still sums to 1 over the full vocabulary of 1446 words, so whatever the unseen bigrams gained was taken from the seen ones.
3. **The discount is very large for frequent bigrams.** P(want | i) drops from 0.3265 to 0.2081, and the count for 'want to' falls from 608 to about 238 (a 61% cut). The count for 'chinese food' falls from 82 to about 8, a 90% cut.
4. **The discount is heaviest for words with small counts.** The row for 'chinese' (C = 158) is divided by 158 + 1446 = 1604, so V dominates the denominator and the smoothed row becomes nearly flat. The row for 'i' (C = 2533) is changed less in relative terms.
5. **Add-1 is a blunt tool.** It adds the same amount to every unseen bigram, whether it is plausible (for example 'eat lunch' if it had been unseen) or impossible (for example 'to to'). It cannot tell them apart.

Limitations and better alternatives

Because the vocabulary is large and most bigrams are unseen, add-1 smoothing takes too much probability from the observed events. For this reason it is rarely used for N-gram language models, although it is still used for tasks with fewer zeros, such as text classification. Better methods include:

- **Add-k smoothing:** add a fractional count k (such as 0.05 or 0.5) instead of 1, so less mass is moved.
- **Backoff:** if the trigram count is zero, use the bigram; if that is zero, use the unigram (Katz backoff, Stupid backoff).
- **Interpolation:** always mix unigram, bigram and trigram estimates with weights λ that sum to 1.
- **Good-Turing discounting:** use the number of words seen once to estimate the probability of unseen words.
- **Kneser-Ney smoothing:** use the number of different contexts a word appears in, which is the best-performing method for N-gram models in practice.

Question 2 – Bag Generation Using an N-gram Grammar

Question. Suppose someone took all the words in a sentence and reordered them randomly. Write a program which takes such a bag of words as input and produces as output a guess at the original order. You will need an N-gram grammar produced by your N-gram program (on some corpus). This task is sometimes called *bag generation*.

Problem definition

Given an unordered multiset of words B = {w1, w2, ..., wn}, find the ordering (permutation) of B that a language model considers the most probable sentence:

ordering* = argmax over all permutations π of B of P(wπ(1) wπ(2) ... wπ(n))

Bag generation is a classic test of a language model. It is also the core of many real tasks: the word-ordering step of statistical machine translation, natural language generation, and re-ordering the output of a speech recogniser or an OCR system.

Why an N-gram model helps

A bag has no ordering information at all, so the only knowledge that can recover the order is knowledge about **which word sequences are likely in the language**. An N-gram language model supplies exactly this. For example, a bigram model trained on restaurant queries will give a high probability to 'want to', 'to eat' and 'eat chinese', and a very low probability to 'to want' or 'food i'. Multiplying these local probabilities along a candidate sentence gives a score, and the sentence with the best score is our guess.

With a bigram model the score of a candidate sentence w1 ... wn is

score = P(w1 | <s>) · P(w2 | w1) · ... · P(wn | wn-1) · P(</s> | wn)

where <s> and </s> are the sentence-start and sentence-end markers. In practice we add log probabilities instead of multiplying, to avoid numerical underflow. Smoothing is essential, because a candidate ordering will almost always contain a bigram that was never seen in training and an unsmoothed model would give it probability zero. Ties between many zero-probability orderings would make the output meaningless.

Design of the program

The program has three components.

1. **N-gram trainer.** Reads a corpus, pads each sentence with N-1 start markers <s> and one end marker </s>, and counts every N-gram and every history (context). It stores the vocabulary size V.
2. **Smoothed probability function.** Uses add-k smoothing (k = 0.05, a small fractional version of the add-1 method used in Question 1): P(w | history) = (C(history, w) + k) / (C(history) + k·V). A small k moves much less probability mass than add-1.
3. **Search procedure.** Finds the best permutation. Two versions are implemented:
    - **Exhaustive search**, which scores every distinct permutation. It is exact, but there are n! permutations, so it is feasible only for short bags (up to about 8 or 9 words).
    - **Beam search**, which builds the sentence left to right and keeps only the best W partial sentences (the beam width, here W = 50) at every step. It is approximate, but its cost grows only polynomially with n.

Algorithm

Training

1. For each corpus sentence, form the token list: (N-1) copies of <s>, the words, then </s>.
2. For every position i, increment count(history, word) and count(history).
3. Record all words in the vocabulary V.

Beam search generation

1. Initialise the beam with one hypothesis: empty sequence, the full bag as remaining words, score 0.
2. Repeat n times: for each hypothesis in the beam and for each distinct word still left in its remaining bag, append the word, remove it from the bag and add log P(word | last N-1 words) to the score.

3. Merge identical hypotheses and keep only the W highest-scoring ones.
4. After n steps add log P(</s> | last N-1 words) to each finished hypothesis.
5. Return the hypothesis with the highest total score.

Complexity

Method	Number of candidates scored	Exact?	Practical size
Exhaustive	n! (fewer if words repeat)	Yes	up to about 8–9 words
Beam search (width W)	about n × n × W	No, approximate	dozens of words
Greedy (W = 1)	about n × n	No, may get stuck	any length

For n = 10 words there are 3,628,800 permutations, whereas beam search with W = 50 scores only about 10 × 10 × 50 = 5,000 extensions.

Python program

```
import math, itertools, random
from collections import Counter

CORPUS = '''i want to eat chinese food
i want to eat lunch
i want to spend money
i want chinese food
i would like to eat lunch
tell me about chinese food
i want to eat dinner
i am looking for a cheap restaurant
i want to eat at a cheap restaurant
can you tell me about a good restaurant
i would like to spend less money
the restaurant serves chinese food
i want to eat british food
give me a cheap lunch
i want lunch
what is a good chinese restaurant
i want to have lunch at a restaurant
please tell me about cheap food'''.split('\n')

class NGram:
    def __init__(self, n=2, k=0.05):
        self.n, self.k = n, k
        self.ctx, self.grams = Counter(), Counter()
        self.vocab = set(['</s>'])

    def train(self, sents):
        for s in sents:
            toks = ['<s>'] * (self.n - 1) + s.split() + ['</s>']
            self.vocab.update(s.split())
            for i in range(self.n - 1, len(toks)):
                h = tuple(toks[i - self.n + 1:i])
```

```
                self.grams[h + (toks[i],)] += 1
                self.ctx[h] += 1

    def prob(self, h, w):                       # add-k smoothing
        V = len(self.vocab)
        return (self.grams[h + (w,)] + self.k) / (self.ctx[h] + self.k * V)

    def logp(self, seq):                        # log P(sentence)
        toks = ['<s>'] * (self.n - 1) + list(seq) + ['</s>']
        return sum(math.log(self.prob(tuple(toks[i - self.n + 1:i]), toks[i]))
                   for i in range(self.n - 1, len(toks)))

def exhaustive(bag, lm):                        # exact, n! permutations
    best, best_score = None, -1e18
    for p in set(itertools.permutations(bag)):
        s = lm.logp(p)
        if s > best_score:
            best, best_score = p, s
    return list(best), best_score

def beam(bag, lm, width=50):                    # approximate, left to right
    beams = [((), Counter(bag), 0.0)]
    for _ in range(len(bag)):
        nxt = {}
        for seq, rem, sc in beams:
            h = (['<s>'] * (lm.n - 1) + list(seq))[-(lm.n - 1):]
            for w in list(rem):
                r = rem.copy(); r[w] -= 1
                if r[w] == 0: del r[w]
                ns = sc + math.log(lm.prob(tuple(h), w))
                nxt[(seq + (w,), tuple(sorted(r.items())))] = (seq + (w,), r, ns)
        beams = sorted(nxt.values(), key=lambda x: -x[2])[:width]
    fin = [(s, sc + math.log(lm.prob(tuple((['<s>'] * (lm.n - 1) + list(s))[-(lm.n -
1):]), '</s>')))
           for s, _, sc in beams]
    s, sc = max(fin, key=lambda x: x[1])
    return list(s), sc

if __name__ == '__main__':
    random.seed(7)
    for n in (2, 3):
        lm = NGram(n); lm.train(CORPUS)
        print(f'--- {n}-gram model ---')
        for orig in ['i want to eat chinese food', 'tell me about a good
restaurant']:
            bag = orig.split(); random.shuffle(bag)      # make the bag
            guess, _ = beam(bag, lm)
            print('bag   :', bag)
            print('guess :', ' '.join(guess), '| correct:', ' '.join(guess) == orig)
```

2.6 How to run and the training corpus

Save the program as bag.py and run **python bag.py**. The corpus used here is a small set of 18 restaurant-domain sentences in the style of the Berkeley Restaurant Project (the same domain as Question 1). To

use a bigger corpus (for example the Brown corpus), replace CORPUS with a list of one sentence per string, lower-cased and tokenised. The program needs no external libraries.

Sample runs (actual output of the program)

Each original sentence was shuffled with random.seed(7) to make the bag, and both search methods were run.

Shuffled bag (input)	Model	Output of the program	Correct?
chinese, i, food, eat, want, to	Bigram	i want to eat chinese food	Yes
about, a, me, restaurant, good, tell	Bigram	tell me about a good restaurant	Yes
to, like, less, money, i, would, spend	Bigram	i would like to spend less money	Yes
a, cheap, lunch, me, give	Bigram	give me a cheap lunch	Yes
want, to, eat, food, chinese, i	Trigram	i want to eat chinese food	Yes
me, about, tell, a, restaurant, good	Trigram	tell me about a good restaurant	Yes
to, money, less, like, would, spend, i	Trigram	i would like to spend less money	Yes
cheap, me, a, give, lunch	Trigram	give me a cheap lunch	Yes

In every run the exhaustive search and the beam search returned the same ordering, which shows that a beam width of 50 was enough for bags of this size.

Test on sentences that are not in the corpus. The results above are optimistic because those four sentences were part of the training corpus. A fairer test uses new sentences built from the same vocabulary:

Original (unseen) sentence	Model	Output	Correct?
i want to eat at a good restaurant	Bigram	i want to eat at a good restaurant	Yes
please give me a cheap chinese lunch	Bigram	please give me a cheap chinese lunch	Yes
tell me about british food	Bigram	tell me about british food	Yes
i want to eat at a good restaurant	Trigram	i want to eat at a good restaurant	Yes
please give me a cheap chinese lunch	Trigram	give me a cheap lunch chinese please	**No**
tell me about british food	Trigram	tell me about british food	Yes

Analysis of the results

1. **Local word order is learned well.** Pairs such as 'want to', 'to eat', 'tell me', 'me about', 'a cheap' and 'a good' are frequent in the corpus, so the model strongly prefers them and chains them into a fluent sentence.
2. **Start and end markers help.** The probability P(i | <s>) is high because most training sentences begin with 'i', so the model correctly puts 'i' first. 'please' is rare at the start, but P(please | <s>) is still higher than P(lunch | <s>), which is why it is correctly placed first in the bigram run.
3. **The trigram model failed on one unseen sentence.** The trigram 'a cheap chinese' never occurs in the training data, so after 'a cheap' the smoothed model has almost no evidence about the next word, so its estimate is spread nearly evenly over the vocabulary. The trigram model has more parameters than a tiny corpus can support, which is the sparsity problem of Question 1 again. With a bigger corpus the trigram model would normally be better than the bigram model.
4. **Search error versus model error.** When the output is wrong, either the model scores the wrong sentence higher than the right one (a *model error*) or the search never found the best-scoring sentence (a *search error*). For the failure above we compared scores: exhaustive search found an ordering with log-probability -18.45, while the correct sentence scores only -24.09. The model genuinely prefers the wrong order, so this is a model error caused by the small corpus and not a search error.

Limitations and possible improvements

- **Several orderings can be grammatical.** 'Yesterday i ate lunch' and 'i ate lunch yesterday' are both fine, so the 'original' order cannot always be recovered, even by a perfect language model.
- **Corpus size and domain.** A model trained on restaurant queries will do poorly on news text. A large, general corpus gives much better results.
- **Better smoothing.** Replacing add-k by interpolated Kneser-Ney or Katz backoff improves the probabilities of unseen N-grams.
- **Longer bags.** For bags of 15 or more words, beam search with a larger width, or an A* search with an admissible estimate of the remaining words, is needed.
- **Neural models.** Modern systems score candidate orders with neural language models, which handle long-distance agreement (for example, 'the keys to the cabinet are') better than N-grams.

Question 3 – Penn Treebank Tagging of Damon Runyon Sentences

Question. Use the Penn Treebank tagset to tag each word in the following sentences from Damon Runyon's short stories. You may ignore punctuation. Some of these are quite difficult; do your best.

Part-of-speech tagging and the Penn Treebank tagset

Part-of-speech (POS) tagging is the task of assigning to each word in a sentence its grammatical category (noun, verb, adjective and so on). It is an early step in almost every NLP pipeline (parsing, information extraction, speech synthesis, machine translation) and it is a hard problem because many words are **ambiguous**: 'back' can be a noun, adjective, adverb or verb, and 'sells' is a verb here but 'a sells' would never occur. The right tag is decided by the word's **context**.

The **Penn Treebank (PTB) tagset** has 45 tags (36 word tags and 9 punctuation tags). The tags used in this answer are listed below.

Tag	Meaning	Example
CC	Coordinating conjunction	and, but, or
DT	Determiner	a, the, this, each
EX	Existential 'there'	there (in 'there is')
IN	Preposition or subordinating conjunction	in, over, of, at, with
JJ	Adjective	nice, tall, sad
NN	Noun, singular or mass	night, game, voice, nobody
NNS	Noun, plural	newspapers, peeks
NNP	Proper noun, singular	Mindy, Street
POS	Possessive ending	's
PRP	Personal pronoun	it, he, she, I, you
RB	Adverb	ever, very, indeed, back, there
RP	Particle	on (in 'put on'), up, off
TO	The word 'to'	to
UH	Interjection	why, oh, well
VB	Verb, base form	take, be
VBG	Verb, gerund or present participle	sitting, putting, taking
VBP	Verb, non-3rd person singular present	am, are, get
VBZ	Verb, 3rd person singular present	is, takes, sells
WDT	Wh-determiner	which, that
WRB	Wh-adverb	when, where, why, how

Method

Each sentence was tagged in three steps: (1) tokenise the sentence, splitting clitics such as 's from the word before them as the Penn Treebank does; (2) list the possible tags of each word; (3) choose the tag that fits the neighbouring words and the whole clause, using the rules of the PTB tagging guidelines (for example, 'to' is always tagged TO, and indefinite pronouns such as 'nobody' are tagged NN). Punctuation is ignored, as the question allows.

Sentence (a): It is a nice night.

Word	Tag	Reason
It	PRP	Personal pronoun, subject of the verb
is	VBZ	Present tense of 'be', third person singular
a	DT	Indefinite article
nice	JJ	Adjective modifying 'night'
night	NN	Singular common noun

Tagged: It/PRP is/VBZ a/DT nice/JJ night/NN

Sentence (b): This crap game is over a garage in Fifty-second Street.

Word	Tag	Reason
This	DT	Determiner modifying the noun group 'crap game'
crap	NN	Noun used as a modifier ('crap' is the dice game); in noun-noun compounds the first noun is tagged NN
game	NN	Head noun of the subject
is	VBZ	Present tense of 'be'
over	IN	Preposition meaning 'above' (a location)
a	DT	Indefinite article
garage	NN	Object of the preposition 'over'
in	IN	Preposition
Fifty-second	JJ	Hyphenated ordinal; it describes the street, so it is tagged as an adjective
Street	NNP	Part of a proper name, so it is capitalised and tagged NNP

Tagged: This/DT crap/NN game/NN is/VBZ over/IN a/DT garage/NN in/IN Fifty-second/JJ Street/NNP

Difficulty: 'crap' looks like an offensive adjective, but here it is part of the compound noun 'crap game'. 'Fifty-second Street' is a name, and some annotators tag both words NNP; tagging the ordinal as JJ is the usual Treebank choice for ordinal numbers that modify a noun.

Sentence (c): Nobody ever takes the newspapers she sells.

Word	Tag	Reason
Nobody	NN	Indefinite pronoun; the PTB guidelines tag 'nobody', 'somebody' and 'everybody' as NN

Word	Tag	Reason
ever	RB	Adverb of time
takes	VBZ	Main verb, third person singular present
the	DT	Definite article
newspapers	NNS	Plural common noun, object of 'takes'
she	PRP	Personal pronoun, subject of 'sells'
sells	VBZ	Verb of the relative clause 'she sells' (the relative pronoun 'that' or 'which' is omitted, so the object 'newspapers' is understood)

Tagged: Nobody/NN ever/RB takes/VBZ the/DT newspapers/NNS she/PRP sells/VBZ

Difficulty: 'newspapers she sells' is a **reduced relative clause**. A tagger that does not notice the missing relative pronoun may take 'newspapers she' as a noun group, so 'sells' must be recognised as a verb because it follows a subject pronoun.

Sentence (d): He is a tall, skinny guy with a long, sad, mean-looking kisser, and a mournful voice.

Word	Tag	Reason
He	PRP	Personal pronoun
is	VBZ	Present tense of 'be'
a	DT	Indefinite article
tall	JJ	Adjective
skinny	JJ	Adjective; a second adjective in a comma-separated list
guy	NN	Head noun
with	IN	Preposition
a	DT	Indefinite article
long	JJ	Adjective
sad	JJ	Adjective
mean-looking	JJ	Hyphenated adjective built from 'mean' + 'looking'; the whole word behaves as one adjective before the noun
kisser	NN	Slang for 'face'; common noun, object of 'with'
and	CC	Coordinating conjunction joining two noun phrases

Word	Tag	Reason
a	DT	Indefinite article
mournful	JJ	Adjective
voice	NN	Common noun

Tagged: He/PRP is/VBZ a/DT tall/JJ skinny/JJ guy/NN with/IN a/DT long/JJ sad/JJ mean-looking/JJ kisser/NN and/CC a/DT mournful/JJ voice/NN

Difficulty: 'kisser' is slang and is not a word an ordinary tagger would have seen as a noun. 'mean-looking' ends in '-ing' and so might be mistaken for VBG, but it is an adjective modifying 'kisser'.

Sentence (e): I am sitting in Mindy's restaurant putting on the gefillte fish, which is a dish I am very fond of.

Word	Tag	Reason
I	PRP	Personal pronoun, subject
am	VBP	Present tense of 'be', first person (non-third-person singular)
sitting	VBG	Present participle in the progressive 'am sitting'
in	IN	Preposition
Mindy	NNP	Proper name
's	POS	Possessive ending, split off from 'Mindy' as the Treebank does
restaurant	NN	Common noun
putting	VBG	Present participle (an adverbial participle clause: 'while putting on...')
on	RP	Particle of the phrasal verb 'put on' (here meaning 'to eat'); it is not a preposition because it can move: 'putting the fish on'
the	DT	Definite article
gefillte	JJ	Yiddish word ('stuffed') modifying 'fish'; JJ is the best fit for a word that describes a noun (FW, foreign word, is also defensible)
fish	NN	Mass noun
which	WDT	Relative pronoun introducing a clause that describes 'the gefillte fish'
is	VBZ	Present tense of 'be'
a	DT	Indefinite article
dish	NN	Common noun

Word	Tag	Reason
I	PRP	Personal pronoun, subject of the relative clause
am	VBP	Present tense of 'be'
very	RB	Adverb modifying 'fond'
fond	JJ	Predicate adjective
of	IN	Preposition required by 'fond of'; its object (the relative pronoun) is understood

Tagged: I/PRP am/VBP sitting/VBG in/IN Mindy/NNP 's/POS restaurant/NN putting/VBG on/RP the/DT gefillte/JJ fish/NN which/WDT is/VBZ a/DT dish/NN I/PRP am/VBP very/RB fond/JJ of/IN

Difficulty: This is the hardest sentence. The phrase 'putting on' is a phrasal verb with an idiomatic meaning, so 'on' must be RP and not IN. 'gefillte' is foreign. The tags of 'am' differ from 'is' (VBP versus VBZ) because of the subject's person. The preposition 'of' is stranded at the end of the relative clause, with no noun after it.

Sentence (f): When a guy and a doll get to taking peeks back and forth at each other, why there you are indeed.

Word	Tag	Reason
When	WRB	Wh-adverb introducing a subordinate clause of time
a	DT	Indefinite article
guy	NN	Common noun
and	CC	Coordinating conjunction joining two nouns
a	DT	Indefinite article
doll	NN	Slang for 'woman'; common noun
get	VBP	Plural subject ('a guy and a doll'), so non-third-person present
to	TO	Always tagged TO in the Treebank, even before a gerund
taking	VBG	Gerund after 'get to' ('start doing')
peeks	NNS	Plural noun (a 'peek' is a quick look), object of 'taking'
back	RB	Adverb of direction
and	CC	Conjunction joining two adverbs
forth	RB	Adverb of direction; 'back and forth' is a fixed phrase
at	IN	Preposition

Word	Tag	Reason
each	DT	Determiner in the reciprocal 'each other'
other	JJ	Treebank convention for 'each other' is DT JJ
why	UH	Interjection at the start of the main clause ('why, there you are'); it is not a question word here
there	RB	Locative adverb ('you are there'), not existential 'there', because 'there' is not followed by a subject-less 'be' clause
you	PRP	Personal pronoun, subject
are	VBP	Present tense of 'be', second person
indeed	RB	Adverb of emphasis

Tagged: When/WRB a/DT guy/NN and/CC a/DT doll/NN get/VBP to/TO taking/VBG peeks/NNS back/RB and/CC forth/RB at/IN each/DT other/JJ why/UH there/RB you/PRP are/VBP indeed/RB

Difficulty: 'get to taking' is ambiguous: 'to' could be an infinitive marker or a preposition, but the Treebank gives it TO in both cases. 'why' is usually WRB, and a tagger has to notice that it is an interjection here. 'there' could be EX, but 'there you are' is a locative use. 'back' can be NN, JJ, RB or VB; here it is an adverb.

Summary of the tagged sentences

Sentence	Words	Tagged output
(a)	5	It/PRP is/VBZ a/DT nice/JJ night/NN
(b)	10	This/DT crap/NN game/NN is/VBZ over/IN a/DT garage/NN in/IN Fifty-second/JJ Street/NNP
(c)	7	Nobody/NN ever/RB takes/VBZ the/DT newspapers/NNS she/PRP sells/VBZ
(d)	16	He/PRP is/VBZ a/DT tall/JJ skinny/JJ guy/NN with/IN a/DT long/JJ sad/JJ mean-looking/JJ kisser/NN and/CC a/DT mournful/JJ voice/NN
(e)	21	I/PRP am/VBP sitting/VBG in/IN Mindy/NNP 's/POS restaurant/NN putting/VBG on/RP the/DT gefillte/JJ fish/NN which/WDT is/VBZ a/DT dish/NN I/PRP am/VBP very/RB fond/JJ of/IN
(f)	21	When/WRB a/DT guy/NN and/CC a/DT doll/NN get/VBP to/TO taking/VBG peeks/NNS back/RB and/CC forth/RB at/IN each/DT other/JJ why/UH there/RB you/PRP are/VBP indeed/RB

Why these sentences are difficult for a tagger

1. **Slang and unusual words.** Runyon writes in a street-slang style ('doll', 'kisser', 'crap game', 'peeks'), which a tagger trained on newspaper text rarely sees.
2. **Ambiguity of form.** 'back', 'on', 'over', 'that', 'to' and 'why' each have several possible tags and need context to be resolved.
3. **Phrasal verbs.** 'put on' and 'get to' have meanings that are not the sum of their parts, and the Treebank must decide between RP and IN for the particle.
4. **Omitted words.** 'the newspapers she sells' leaves out 'that', and 'fond of' leaves out the object.
5. **Hyphenated and foreign words.** 'mean-looking', 'Fifty-second' and 'gefillte' are rare, so a statistical tagger would have to guess from suffixes and capitalisation.